

News Analyst Agent

Multi-Step LLM Pipeline — Written Report

1. Problem Statement

Staying informed on a fast-moving news topic requires more than a single search or a one-shot question to a language model. A single prompt asking "what is happening with X?" forces the model to rely entirely on training data, which may be months out of date, and produces a flat, undifferentiated answer that conflates recent events, background context, expert opinion, and future implications into a single paragraph. The result is often vague, poorly sourced, and difficult to act on.

The News Analyst Agent solves this by decomposing the task into six interdependent steps. The agent accepts a news topic from the user and produces a structured, analyst-style intelligence briefing backed by real web sources retrieved during the run. Multi-step chaining is not optional here — it is load-bearing. The quality of the final briefing depends directly on the accuracy of the search queries (Step 1), the breadth of the retrieved articles (Step 2), the uniformity of the per-article summaries (Step 3), the cross-article pattern recognition (Step 4), the structured drafting (Step 5), and the editorial refinement (Step 6). Collapsing any two consecutive steps into one produces a measurably worse output because each step performs a cognitively distinct task that benefits from focused, constrained prompting.

2. Chain Design

Step 1 — Query Planner (LLM)

Receives: The raw user topic string. **Produces:** A JSON object containing four targeted search queries and their angle labels (recent news, background, expert opinion, future implications).

A single broad search on the raw topic would retrieve articles that cluster around the most popular angle — usually recent news — and miss background context, expert commentary, and forward-looking analysis. This step uses the LLM to think strategically about what a well-rounded investigation requires before any search is made. The output is a rigid JSON schema so that Step 2 can iterate over the query list programmatically without any parsing ambiguity.

Step 2 — Web Search (Tool Call, Serper API)

Receives: The four queries from Step 1. **Produces:** A flat, deduplicated list of article dicts containing title, snippet, source URL, and publication date.

This is the only step that retrieves information from outside the model. LLMs hallucinate recent facts with confidence; a tool call grounds the rest of the chain in real, dated content. Each query is executed independently, and failures are caught per-query — if one search returns nothing, the other three continue unaffected. Results are deduplicated by URL before being passed forward.

Step 3 — Article Summariser (LLM)

Receives: The flat article list from Step 2. **Produces:** A structured JSON array where every article is normalised to the same schema: headline, key_facts[], sentiment, source_name.

The raw Serper snippets are inconsistent in length, relevance, and format. Step 4 needs to reason across all articles simultaneously, which is only practical if the inputs are uniform. This step is a

normalisation pass: it extracts only what matters from each article and discards noise. A fallback is included — if JSON parsing fails, minimal summaries are constructed from the raw fields so the chain does not break.

Step 4 — Theme Extractor (LLM)

Receives: The normalised summaries from Step 3. **Produces:** A theme analysis object containing 3–5 named themes, contradictions between sources, consensus, overall sentiment, and key named entities.

This is the analytical core of the pipeline. Where Step 3 reasons about individual articles, Step 4 reasons across all of them. It identifies patterns that are invisible in any single article: recurring themes, conflicting narratives, and what all sources agree on. This higher-order analysis is exactly what a briefing writer needs as input — a structured understanding of the coverage landscape rather than a flat list of facts.

Step 5 — Briefing Writer (LLM)

Receives: The theme analysis from Step 4, the summaries from Step 3, and the topic. **Produces:** A structured Markdown briefing with fixed sections: Executive Summary, Key Themes, Points of Tension, Key Entities, and Sources.

Synthesis and writing are kept as separate steps because they require different modes of reasoning. A single call asked to both find themes and write a polished document produces worse output than a dedicated writer that receives already-structured theme data and focuses solely on prose quality. The system prompt enforces a fixed Markdown template, which ensures the output is predictable enough for the Step 6 critic to assess against a known structure.

Step 6 — Critic and Refiner (LLM x2)

Receives: The draft briefing from Step 5. **Produces:** A JSON critique object (weaknesses, missing angles, bias flags) and a final polished briefing with an appended Editorial Notes section.

The draft from Step 5 was written to cover the given data as completely as possible. A separate critic is needed because the writer cannot reliably identify its own gaps — it has already committed to a framing. The critic call returns structured feedback that the refiner then addresses specifically, producing a qualitatively better output than any single draft-and-publish approach.

3. Tool Integration

Tool used: Serper API (<https://serper.dev>) — free tier, 2,500 searches per month.

Serper was chosen over alternatives such as Tavily or DuckDuckGo for three reasons: its free tier requires no credit card, its response format is a clean JSON object with an 'organic' array of results, and it reliably returns publication dates alongside snippets — which are important for time-sensitive news topics. The Serper call is isolated in `tools.py` so that switching to a different search provider requires changing only one function.

The tool output enters the chain as a flat list of article dicts passed directly into Step 3's user prompt. Each dict contains title, snippet, source URL, and date. Step 3 receives this as a numbered article block and returns the same articles in a normalised schema. The tool's output is never used directly by an LLM without this normalisation pass — the raw snippets are too inconsistent in length and structure for reliable downstream reasoning.

4. Limitations

Several limitations were identified during testing and could not be fully addressed within the scope of this assignment.

Snippet-only content. Serper returns article snippets rather than full text. The agent analyses summaries of 2–3 sentence excerpts, which means nuance, data, and extended argument are frequently lost. A production version would fetch the full HTML of each article URL.

Four-query breadth cap. Only 16 articles are retrieved per run (4 queries × 4 results). For niche or rapidly evolving topics, this may not be sufficient to identify genuine cross-cutting themes. The number of queries and results per query are easily configurable in the code but were kept low to stay within free-tier rate limits.

No source credibility weighting. A tabloid, a press release, and a Reuters article are treated identically. The theme extractor has no way to distinguish between these, which can skew the consensus and contradiction analysis.

Step 3 JSON fragility. If the LLM returns malformed JSON at Step 3 — which occurs occasionally when the article list is very long — the fallback summaries lose the `key_facts` structure, degrading Step 4's theme quality. A retry-with-simplified-prompt mechanism would address this but was not implemented.

English only. All queries, prompts, and expected outputs are in English. Topics with significant non-English coverage (e.g. regional politics) will be poorly covered.

5. Reflection

The most non-obvious finding during development was that prompt structure matters more than prompt length. Early versions of the Step 3 and Step 4 prompts were verbose and instructional, but produced inconsistent JSON because the model spent tokens interpreting the instructions rather than executing the task. Replacing lengthy descriptions with a concrete JSON schema example in the prompt — showing exactly what one output object should look like — produced consistent, parseable output far more reliably than prose instructions.

A second finding was that separating critique from revision (Step 6's two sub-calls) produced noticeably better final briefings than a single self-edit prompt. When asked to critique and revise in one call, the model consistently underweighted the critique and made only cosmetic changes. Separating them forces the model to commit to a specific list of weaknesses before it writes the revision.

Given more time, the most valuable addition would be full article fetching — retrieving the complete HTML of each search result URL rather than relying on snippets. This alone would dramatically improve the quality of the Step 3 summaries and, by extension, every downstream step. A secondary improvement would be a confidence-scoring mechanism in Step 4 that flags themes supported by only one source as tentative rather than presenting them with the same weight as themes corroborated across multiple sources.